JULY 15, 2026 / #NODE.JS

How to Fix Common Web Application Security Vulnerabilities in Node.js

C Hackita

Here's something that tends to surprise developers who are new to security: most web vulnerabilities aren't the result of sophisticated attacks. They come from code patterns that look completely reasonable: trusting a value from the URL, applying a request body to a database update, or running two queries where one should've been enough.

This guide covers six of those patterns. For each one, you'll see a real code example that creates the vulnerability, an explanation of what makes it dangerous, and a corrected version with notes on exactly what changed and why.

No security background is required to follow along here. It'll just help to have some familiarity with Node.js and SQL.

Prerequisites

The examples assume you're comfortable with:

- Node.js and Express.js basics
- SQL queries
- How HTTP requests and responses work
- Basic authentication concepts (sessions, tokens)

Note on code examples: Throughout this tutorial, `db.query()` is a fictional database helper that returns a single row object for `SELECT` queries (or `null` if not found), and a result object for `INSERT` / `UPDATE` queries. The `connection.query()` in the race conditions section uses the mysql2 promise API directly, where `query()` returns `[rows, fields]`. Adapt the syntax to the database driver you use.

Table of Contents

1. Broken Access Control and IDOR

Broken Access Control has topped the OWASP Top 10 since 2021, and it's not hard to see why. The most common form is **Insecure Direct Object Reference (IDOR)**: the application exposes a database ID in a URL, a user changes the number, and suddenly they're looking at someone else's data.

The fix seems obvious in hindsight. But it keeps appearing in production code because authentication and authorization get conflated. Confirming that a user is logged in is not the same as confirming they're allowed to access a specific resource.

How to Identify This Vulnerability in Your Code

Here's a typical user profile endpoint:

```
// Express.js - Vulnerable
app.get('/api/users/:id/profile', authenticate, async (req, res) => {
  const userId = req.params.id;

  const user = await db.query(
    'SELECT id, name, email, address FROM users WHERE id = ?',
    [userId]
  );

  if (!user) {
    return res.status(404).json({ error: 'User not found' });
  }
```

```
  res.json(user);
});
```

The `authenticate` middleware confirms that the request includes a valid token. But it doesn't confirm whether the authenticated user is allowed to access the requested resource.

Any authenticated user can request `/api/users/1/profile`, `/api/users/2/profile`, and so on and retrieve other users' data.

Why This Matters

Authentication confirms *who* you are. Authorization confirms *what you're allowed to do*. The code above does the first and skips the second entirely.

With a sequential numeric ID, a curious user doesn't need any special tools. They can just change `1` to `2` in the URL. But the same problem exists with UUIDs or slugs if the ownership check is missing.

How to Fix This Vulnerability

Verify server-side that the authenticated user owns — or is explicitly authorized to access — the requested resource:

```
// Express.js - Secure
app.get('/api/users/:id/profile', authenticate, async (req, res) => {
  // Reject anything that isn't a string of digits — parseInt("12abc")
  if (!/^\d+$/.test(req.params.id)) {
    return res.status(400).json({ error: 'Invalid user ID' });
  }

  const requestedId = Number(req.params.id);

  if (requestedId < 1) {
```

```
    return res.status(400).json({ error: 'Invalid user ID' });
  }

  // The authenticated user's ID is set by the authenticate middleware
  const authenticatedId = req.user.id;

  // Enforce ownership: users can only access their own profile
  if (requestedId !== authenticatedId) {
    return res.status(403).json({ error: 'Forbidden' });
  }

  const user = await db.query(
    'SELECT id, name, email, address FROM users WHERE id = ?',
    [requestedId]
  );

  if (!user) {
    return res.status(404).json({ error: 'User not found' });
  }

  res.json(user);
});
```

For admin endpoints that legitimately need to access any user, enforce role-based authorization explicitly:

```
// Admin endpoint with explicit role check
app.get('/api/admin/users/:id', authenticate, requireRole('admin'), asy
  if (!/^\d+$/.test(req.params.id)) {
    return res.status(400).json({ error: 'Invalid user ID' });
  }

  const userId = Number(req.params.id);

  const user = await db.query(
    'SELECT id, name, email, role FROM users WHERE id = ?',
    [userId]
  );

  if (!user) {
    return res.status(404).json({ error: 'User not found' });
  }
```

```
  res.json(user);
});
```

Here's what changed and why:

- `/^\d+$/.test(req.params.id)` rejects anything that isn't a pure string of digits. `parseInt("12abc", 10)` would silently return `12` and pass further checks. The regex prevents this.
- `Number(req.params.id)` converts the already-validated string to a number safely.
- The comparison `requestedId !== authenticatedId` enforces ownership.
- Admin functionality is a separate endpoint with its own authorization check.

Never infer authorization from a URL parameter. Derive it from the authenticated session.

2. Mass Assignment

Mass assignment is one of those vulnerabilities that's almost invisible when you write it. You're just being efficient, right? Why iterate over fields manually when you can pass the whole object?

The problem is that your database table knows about fields your users were never supposed to touch.

How to Identify This Vulnerability in Your Code

Here's a user profile update endpoint:

```
// Express.js - Vulnerable
app.put('/api/users/me', authenticate, async (req, res) => {
  const userId = req.user.id;

  // req.body contains everything the client sends
  const updates = req.body;

  await db.query(
    'UPDATE users SET ? WHERE id = ?',
    [updates, userId]
  );

  res.json({ success: true });
});
```

The `users` table has these columns:

```
CREATE TABLE users (
  id           INT PRIMARY KEY,
  name         VARCHAR(100),
  email        VARCHAR(100),
  bio          TEXT,
  role         ENUM('user', 'moderator', 'admin') DEFAULT 'user',
  credits      INT DEFAULT 0,
  is_banned    BOOLEAN DEFAULT false
);
```

The developer intended users to update `name`, `email`, and `bio`. But `role`, `credits`, and `is_banned` are also in the table — and the query updates whatever fields the client sends.

Why This Matters

That request body goes straight into the SQL query. The `users` table also has `role`, `credits`, and `is_banned` — and the query doesn't know or care which fields the developer "intended" to expose.

A user who sends this:

```
{
  "name": "Alice",
  "role": "admin",
  "credits": 100000,
  "is_banned": false
}
```

Has just promoted themselves to admin, cleared their ban, and given themselves a hundred thousand credits.

How to Fix This Vulnerability

Build the update object yourself, field by field, using an explicit allowlist:

```
// Express.js - Secure
app.put('/api/users/me', authenticate, async (req, res) => {
  const userId = req.user.id;

  // Only these fields may be updated by the user
  const ALLOWED_FIELDS = ['name', 'email', 'bio'];
  const updates = {};

  for (const field of ALLOWED_FIELDS) {
    if (req.body[field] !== undefined) {
      updates[field] = req.body[field];
    }
  }

  if (Object.keys(updates).length === 0) {
    return res.status(400).json({ error: 'No valid fields provided' })
  }

  // Validate individual fields
  // isValidEmail is a simple helper: /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test
  if (updates.email && !isValidEmail(updates.email)) {
    return res.status(400).json({ error: 'Invalid email format' });
```

```
  }

  if (updates.name && (typeof updates.name !== 'string' || updates.name
    return res.status(400).json({ error: 'Name must be a string of 100
  }

  await db.query(
    'UPDATE users SET ? WHERE id = ?',
    [updates, userId]
  );

  res.json({ success: true });
});
```

For admin operations that legitimately update sensitive fields, use a separate endpoint with its own authorization:

```
// Admin-only endpoint for changing user roles
app.put('/api/admin/users/:id/role', authenticate, requireRole('admin'
  if (!/^\d+$/.test(req.params.id)) {
    return res.status(400).json({ error: 'Invalid user ID' });
  }

  const userId = Number(req.params.id);
  const { role } = req.body;

  const VALID_ROLES = ['user', 'moderator', 'admin'];

  if (!VALID_ROLES.includes(role)) {
    return res.status(400).json({ error: 'Invalid role' });
  }

  await db.query(
    'UPDATE users SET role = ? WHERE id = ?',
    [role, userId]
  );

  res.json({ success: true });
});
```

Don't spread `req.body` into a database query. Build the update object field by field.

3. Prototype Pollution

Prototype pollution occurs when untrusted data is merged into an object recursively, allowing an attacker to inject properties into `Object.prototype` (the base object that every plain JavaScript object inherits from).

The OWASP Top 10 covers this under Software and Data Integrity Failures (A08:2021).

How to Identify This Vulnerability in Your Code

A configuration merge utility that processes user-supplied settings:

```
// Vulnerable recursive merge function
function mergeConfig(target, source) {
  for (const key of Object.keys(source)) {
    if (typeof source[key] === 'object' && source[key] !== null) {
      if (!target[key]) target[key] = {};
      mergeConfig(target[key], source[key]); // recursive call
    } else {
      target[key] = source[key]; // triggers __proto__ setter via brac
    }
  }
}

app.post('/api/settings', authenticate, (req, res) => {
  const userSettings = {};
  mergeConfig(userSettings, req.body); // merge untrusted input
  applySettings(userSettings);
  res.json({ success: true });
});
```

Why This Matters

An attacker sends this request body:

```
{
  "__proto__": {
    "isAdmin": true
  }
}
```

The recursive `mergeConfig` function reaches the `__proto__` key and executes `target['__proto__']['isAdmin'] = true`. Because `__proto__` is JavaScript's prototype accessor, this writes directly to `Object.prototype`. After the merge:

```
const anyObject = {};
console.log(anyObject.isAdmin); // true — inherited from Object.prototy
```

Every plain object in the running application now inherits `isAdmin: true`. If any authorization check looks like this:

```
if (user.isAdmin) { /* grant admin access */ }
```

That check now passes for every user, regardless of their actual role.

How to Fix This Vulnerability

Store user state server-side and validate each field individually. Never recursively merge untrusted input:

```
// Express.js - Secure
app.post('/api/settings', authenticate, async (req, res) => {
  // Load current settings from the database — never from the client
  const current = await db.query(
    'SELECT theme, language, notifications FROM user_settings WHERE use
    [req.user.id]
  );

  // Validate each field against an explicit allowlist
  const safeSettings = {
    theme: validateEnum(req.body.theme, ['light', 'dark'], current.ther
    language: validateEnum(req.body.language, ['en', 'es', 'fr', 'de']
    notifications: typeof req.body.notifications === 'boolean'
      ? req.body.notifications
      : current.notifications
  };

  await db.query(
    'UPDATE user_settings SET ? WHERE user_id = ?',
    [safeSettings, req.user.id]
  );

  res.json({ success: true });
});

function validateEnum(value, allowed, defaultValue) {
  return allowed.includes(value) ? value : defaultValue;
}
```

If you need a merge utility, use `Object.create(null)` as the base — it has no prototype, so `__proto__` can't be polluted — and allowlist keys explicitly:

```
// Safe merge: base object with no prototype
function safeMerge(allowedKeys, source) {
  const result = Object.create(null); // no prototype = no pollution po

  for (const key of allowedKeys) {
    if (key in source && typeof source[key] !== 'object') {
      result[key] = source[key];
    }
  }
```

```
  return result;
}
```

Rules:

- Never recursively merge untrusted input into a plain object.
- Store application state server-side. Don't trust clients to carry it.
- Use `Object.create(null)` for data containers that will hold untrusted keys.
- Validate each field by type and allowed values before using it.

4. Race Conditions

Race conditions are tricky because the code is perfectly correct. It's the timing that breaks it. Two requests arrive at almost the same moment, both check the same condition, both see a valid result, and both proceed. The result is something that should only happen once, happening twice.

This is called a **Time-of-Check to Time-of-Use (TOCTOU)** problem: the state you checked is no longer the state you're acting on.

How to Identify This Pattern in Your Code

A single-use coupon redemption endpoint:

```
// Express.js - Vulnerable
app.post('/api/redeem-coupon', authenticate, async (req, res) => {
  const { couponCode } = req.body;
```

```
  const userId = req.user.id;

  // Step 1: Check if the coupon is still valid
  const coupon = await db.query(
    'SELECT id, discount_amount, used FROM coupons WHERE code = ? AND u
    [couponCode]
  );

  if (!coupon) {
    return res.status(400).json({ error: 'Invalid or already used coupo
  }

  // Time gap: another request can pass Step 1 here before Step 3 runs

  // Step 2: Apply the discount
  await applyDiscountToOrder(userId, coupon.discount_amount);

  // Step 3: Mark coupon as used
  await db.query(
    'UPDATE coupons SET used = true, used_by = ? WHERE code = ?',
    [userId, couponCode]
  );

  res.json({ success: true });
});
```

Why This Matters

Two requests arrive with the same coupon code at nearly the same time. Both hit Step 1 before either reaches Step 3. Both read `used = false`. Both apply the discount. One coupon, used twice.

The same window exists anywhere you read-then-write: balance checks before deductions, inventory checks before reservations, vote checks before incrementing. Any of them can be exploited the same way.

How to Fix This Vulnerability

Replace the check-then-act pattern with an atomic operation. An atomic database update guarantees that the condition check and

the write happen as a single indivisible unit:

```
// Express.js - Secure
app.post('/api/redeem-coupon', authenticate, async (req, res) => {
  const { couponCode } = req.body;
  const userId = req.user.id;

  const connection = await db.getConnection();

  try {
    await connection.beginTransaction();

    // Atomic: only one request can update a row where used = false.
    // The database row lock ensures only one request succeeds.
    const [result] = await connection.query(
      `UPDATE coupons
       SET used = true, used_by = ?, used_at = NOW()
       WHERE code = ? AND used = false`,
      [userId, couponCode]
    );

    if (result.affectedRows === 0) {
      await connection.rollback();
      return res.status(400).json({ error: 'Invalid or already used cou
    }

    const [couponRows] = await connection.query(
      'SELECT discount_amount FROM coupons WHERE code = ?',
      [couponCode]
    );
    const coupon = couponRows[0];

    await applyDiscountToOrder(userId, coupon.discount_amount, connect

    await connection.commit();

    res.json({ success: true, discount: coupon.discount_amount });

  } catch (error) {
    await connection.rollback();
    console.error('Coupon redemption failed:', error);
    res.status(500).json({ error: 'Could not process the coupon' });
  } finally {
    connection.release();
```

```
  }
});
```

The key change is `UPDATE ... WHERE code = ? AND used = false`. The database acquires a row lock during the update. Only one concurrent request can succeed. The second request finds `affectedRows = 0` and returns an error — correctly.

Any time you read a value to make a decision before writing — that's a potential race condition. Make the check and the write atomic.

5. Business Logic Flaws

Business logic flaws are the hardest category to catch. Automated scanners won't find them, and code review might miss them too, because the code works exactly as written. The problem is in what the code was designed to do, not how it does it.

The most common form: trusting the client to send sensible numeric values.

How to Identify This Vulnerability in Your Code

An e-commerce checkout endpoint:

```
// Express.js - Vulnerable
app.post('/api/checkout', authenticate, async (req, res) => {
  const { items } = req.body;

  let total = 0;
  const processedItems = [];

  for (const item of items) {
```

```
    const product = await db.query(
      'SELECT id, price FROM products WHERE id = ?',
      [item.productId]
    );

    if (!product) {
      return res.status(400).json({ error: `Product not found: ${item.
    }

    // Trust the quantity value from the client
    const itemTotal = product.price * item.quantity;
    total += itemTotal;

    processedItems.push({ productId: product.id, quantity: item.quanti
  }

  if (total > 100) {
    total = total * 0.9; // 10% discount
  }

  await createOrder(req.user.id, processedItems, total);
  res.json({ success: true, total });
});
```

Why This Matters

Problem 1 — Negative quantity: The code multiplies the server's price by the client's quantity without checking that the quantity is positive. A user sends `"quantity": -5` on an expensive item. Its contribution to the total becomes negative, reducing the overall total.

Problem 2 — Floating point arithmetic: `0.1 + 0.2` in JavaScript is `0.30000000000000004`. Without rounding, financial calculations accumulate errors over time.

Problem 3 — Discount manipulation: If a separate endpoint allows modifying an order after checkout without recalculating the total, a user could add items to earn the discount, then remove items while keeping the discounted price.

How to Fix This Vulnerability

Validate every numeric input and recalculate all totals server-side. Use integer arithmetic (cents) for money to avoid floating point errors:

```
// Express.js - Secure
app.post('/api/checkout', authenticate, async (req, res) => {
  const { items } = req.body;

  if (!Array.isArray(items) || items.length === 0) {
    return res.status(400).json({ error: 'Cart must contain at least or
  }

  if (items.length > 50) {
    return res.status(400).json({ error: 'Cart cannot contain more thar
  }

  let totalCents = 0; // Integer arithmetic avoids floating point erro
  const processedItems = [];

  for (const item of items) {
    if (!/^\d+$/.test(String(item.productId))) {
      return res.status(400).json({ error: `Invalid product ID: ${item
    }

    const productId = Number(item.productId);

    // Validate quantity: must be a string of digits only, between 1 ar
    // parseInt("3abc", 10) returns 3 — we use regex to prevent this
    if (!/^\d+$/.test(String(item.quantity))) {
      return res.status(400).json({
        error: `Invalid quantity for product ${productId}`
      });
    }

    const quantity = Number(item.quantity);

    if (quantity < 1 || quantity > 10) {
      return res.status(400).json({
        error: `Quantity for product ${productId} must be between 1 and
      });
    }

    const product = await db.query(
```

```
        'SELECT id, name, price_cents, stock FROM products WHERE id = ? A
        [productId]
      );

      if (!product) {
        return res.status(400).json({ error: `Product not found: ${produ
      }

      if (product.stock < quantity) {
        return res.status(400).json({
          error: `Insufficient stock for "${product.name}"`
        });
      }

      // Use the server's price — never trust a price from the client
      totalCents += product.price_cents * quantity;

      processedItems.push({
        productId: product.id,
        name: product.name,
        quantity,
        unitPriceCents: product.price_cents
      });
    }

    // Integer arithmetic for the discount calculation
    const discountMultiplier = totalCents > 10000 ? 90 : 100; // 10000 c
    const finalTotalCents = Math.round(totalCents * discountMultiplier /

    await createOrder(req.user.id, processedItems, finalTotalCents);

    res.json({
      success: true,
      total: (finalTotalCents / 100).toFixed(2),
      currency: 'USD'
    });
  });
```

What changed and why:

- Integer (cents) arithmetic eliminates floating point errors. `10000 + 3000` in integer cents is always exact.
- `quantity < 1 || quantity > 10` prevents negative quantities and unreasonably large orders.

- `items.length > 50` prevents oversized requests.
- `product.stock < quantity` ensures the order doesn't exceed available inventory.
- All totals, discounts, and final prices are calculated server-side from server-side prices.

Define the valid range for every numeric input. Recalculate every total server-side. The client isn't a trusted source for prices or quantities.

6. JWT Misconfiguration

JWTs are everywhere in Node.js APIs, and the `jsonwebtoken` library makes them easy to use. That's both good and bad: they're easy to use correctly, but they're also easy to use in ways that look fine until they're not.

The OWASP Top 10 classifies authentication failures under Identification and Authentication Failures (A07:2021). Three misconfigurations show up repeatedly.

How to Identify This Vulnerability in Your Code

Mistake 1 — Algorithm not specified in `verify()`:

```
// Vulnerable
const decoded = jwt.verify(token, process.env.JWT_SECRET);
```

Without an `algorithms` option, some JWT implementations can be tricked into accepting tokens that declare `"alg": "none"` in their header — meaning no signature is required at all.

Mistake 2 — Weak or hardcoded secrets:

```
// Vulnerable
const token = jwt.sign({ userId: user.id }, 'secret');
```

A short or predictable HS256 secret can be brute-forced offline if an attacker obtains a valid token.

Mistake 3 — Sensitive data in the payload:

```
// Vulnerable
const token = jwt.sign({
  userId: user.id,
  passwordHash: user.passwordHash, // never do this
  role: user.role
}, secret);
```

JWT payloads are base64-encoded, not encrypted. Anyone who holds the token can decode the payload and read its contents.

How to Fix These Issues

```
// Secure JWT implementation
const jwt = require('jsonwebtoken');
const crypto = require('crypto');

// Generate a strong secret once and store it as an environment variabl
// node -e "console.log(crypto.randomBytes(64).toString('hex'))"
const JWT_SECRET = process.env.JWT_SECRET;

if (!JWT_SECRET || Buffer.from(JWT_SECRET, 'hex').length < 32) {
  throw new Error('JWT_SECRET must be at least 32 random bytes');
}

function signToken(userId) {
```

```
  return jwt.sign(
    { sub: userId },          // 'sub' is the standard claim for the use
    JWT_SECRET,
    {
      algorithm: 'HS256',     // always declare the algorithm explicitly
      expiresIn: '15m',       // short-lived tokens reduce the window it
      issuer: 'your-app-name',
      audience: 'your-app-name'
    }
  );
}

function verifyToken(token) {
  return jwt.verify(token, JWT_SECRET, {
    algorithms: ['HS256'],    // whitelist only the expected algorithm
    issuer: 'your-app-name',
    audience: 'your-app-name'
  });
}

function authenticate(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'No token provided' });
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = verifyToken(token);
    req.user = { id: decoded.sub };
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}
```

A note on token revocation: JWTs are stateless, meaning your server keeps no record of them. This means you can't immediately invalidate a token on logout or account compromise without extra infrastructure.

Common solutions include short-lived access tokens (15 minutes) paired with refresh tokens stored server-side, or a token denylist in a fast store like Redis. Choose the approach that matches your application's requirements.

JWT security checklist:

- Always pass `algorithms: ['HS256']` to `verify()`.
- Use a secret of at least 32 random bytes generated with `crypto.randomBytes`.
- Set short expiration times.
- Store only non-sensitive identifiers (user ID) in the payload — not emails, passwords, or roles.
- Use standard claims: `sub`, `iss`, `aud`, `exp`.
- Plan your revocation strategy before going to production.

Summary

Here's a quick reference for the six vulnerability categories covered in this tutorial:

VULNERABILITY	ROOT CAUSE	CORE FI
IDOR	Authorization check missing	Verify
Mass Assignment	All request body fields applied to database	Allowl
Prototype Pollution	Recursive merge of untrusted input	Store s
Race Condition	Check-then-act without atomicity	Atomi
Business Logic Flaw	Trusting client-supplied numeric values	Regex
JWT Misconfiguration	No algorithm allowlist, weak secrets	Explici

None of these vulnerabilities requires a clever attacker. They just need code that trusts the wrong thing at the wrong time.

The patterns worth internalizing: always derive authorization from the session, never from the request. Validate every numeric input with a range. Make financial writes atomic. Build your JWT configuration explicitly.

Understanding how to fix these vulnerabilities is one side of the picture. Understanding how they're actually identified and exploited during a real security assessment is the other.

If you want to go deeper on the offensive side — how penetration testers approach web application targets, what they look for, and how they chain multiple flaws together — this guide to web application attack techniques covers that perspective in detail.

Hackita

https://hackita.it
